

UNIVERSIDAD DEL VALLE DE GUATEMALA

Redes

Facultad de Ingeniería

Sección 30

Juan Carlos Canteo Boix



Laboratorio 2 - Esquemas de detección y corrección de errores

Joel Antonio Jaquez López - 23369

Jose Pablo Lopez Lopez - 23773

Enlace al repositorio:

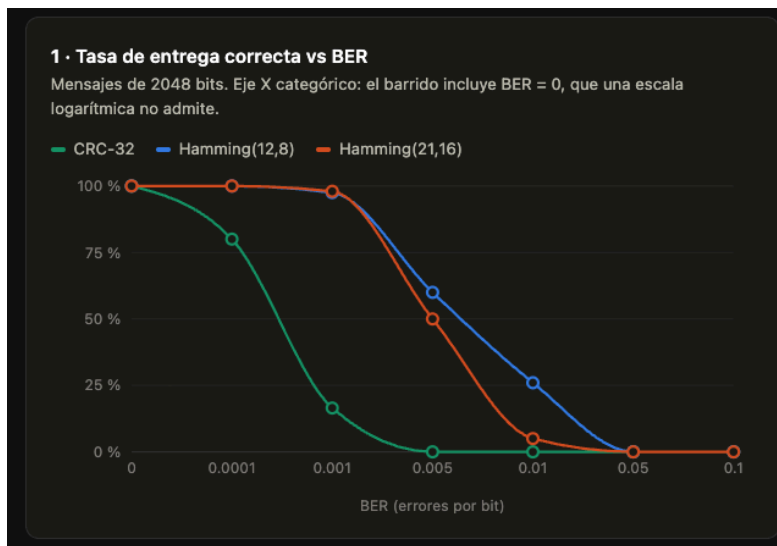
https://github.com/jjaquezito/Laboratorio2_Reddes.git

GUATEMALA, 21 de agosto 2026

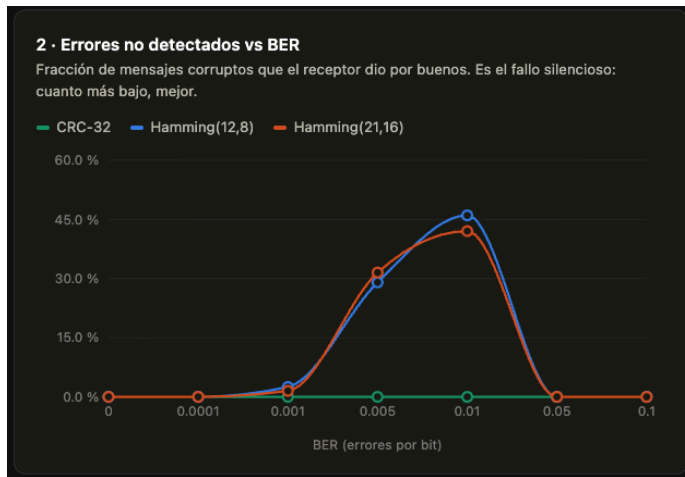
Descripción de la práctica:

Se desarrolló una aplicación de transmisión y recepción de mensajes sobre una arquitectura de 5 capas, que son: Aplicación, Presentación, Enlace, Ruido y Transmisión. Con el fin de analizar el comportamiento de los esquemas de detección y corrección de errores cuando la información viaja por un canal no confiable. El emisor se implementó en Python y el receptor en TypeScript, comunicándose mediante sockets TCP. El lenguaje se codifica en ASCII binario de 8 bits por carácter, la capa de Enlace le concatena información de redundancia calculada con uno de los 2 algoritmos (Hamming para corrección y CRC-32 para detección) y la capa de Ruido simula la interferencia del medio invirtiendo cada bit de la trama de forma independiente con una probabilidad p solicitada al momento del envío, la cual afecta tanto a los datos como a la redundancia. En el receptor, el proceso se invierte porque verifica la integridad, se corrige el error cuando el algoritmo lo permite y se decodifica el mensaje o bien se reporta la falla cuando la recuperación no es posible.

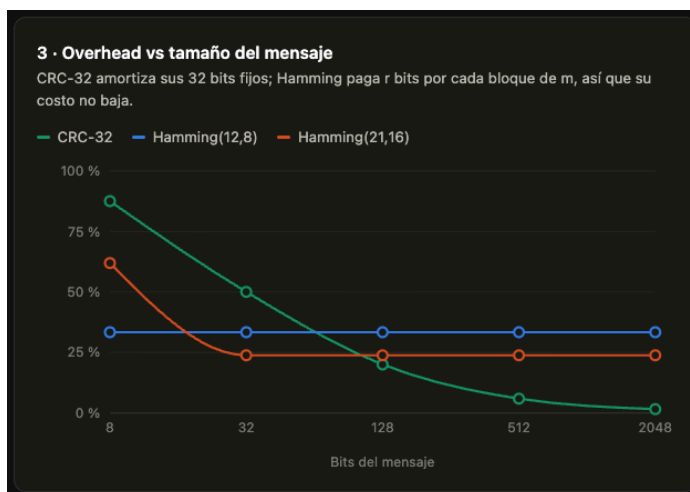
Resultados:



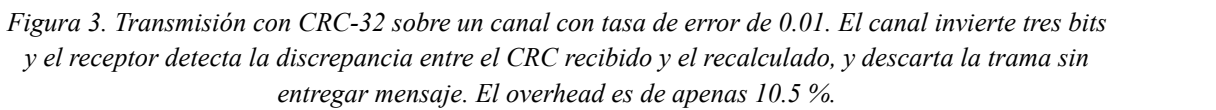
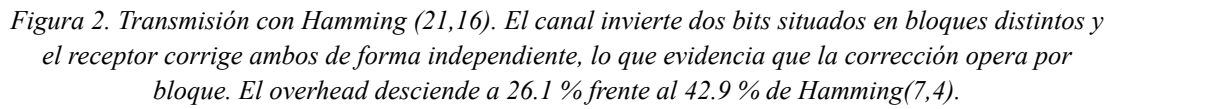
- Esta gráfica representa el porcentaje de mensajes que llegaron al receptor de forma idéntica al original, para cada configuración conforme se degrada la calidad del canal, se utilizaron mensajes de 2048 bits y 300 repeticiones por punto. Los resultados indican que la corrección de errores ofrece una ventaja real pero acotada a una ventana estrecha de operación con una tasa de error del 0.001, como se puede ver en la gráfica Hamming (12,8) entregó correctamente 98.3% de los mensajes frente a apenas 15.0% de CRC-32, esta diferencia se explica porque este último detecta la corrupción pero carece de la capacidad para repararla. A partir de una tasa de 0.01 ambos esquemas colapsan, se observa que Hamming cae al 20.3% y CRC-32 al 0% esto pasa porque en tramas de esa longitud la probabilidad de que algún bloque acumule más errores de los corregibles se aproxima a la certeza. Se observa además que la configuración con $m=8$ supera consistentemente a la de $m=16$, lo cual resulta esperable: los bloques más pequeños concentran una mayor proporción de bits de paridad y, por tanto, mayor capacidad de corrección, a costa de un overhead superior.

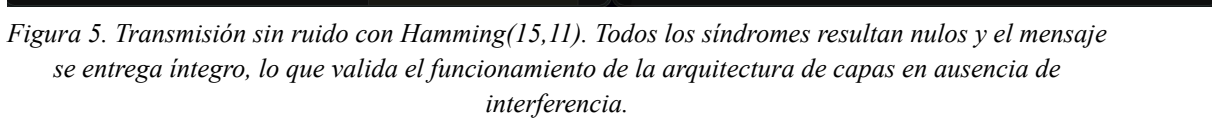
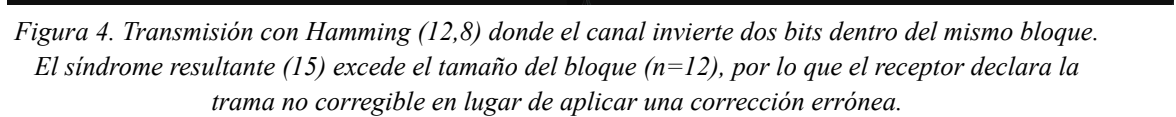


- Esta gráfica representa la fracción de mensajes corruptos que el receptor aceptó como válidos y entregó a la capa de aplicación, se trata de un fallo silencioso, en el que el destinatario recibe información errónea sin advertencia alguna. El resultado más relevante es que CRC-32 registró 0% de falsos negativos en la totalidad del barrido, eso quiere decir que en ninguna de las 10,500 transmisiones evaluadas entregó datos corruptos como si fueran válidos. Por el contrario Hamming alcanzó un 44.7% de falsos negativos con $m=8$ y 46.3% con $m=16$ en una tasa de error de 0.01.



- Esta gráfica representa la proporción de la trama transmitida que corresponde a la información de redundancia y no a datos útiles, en función de la longitud del mensaje original. Los resultados muestran 2 comportamientos opuestos. Hamming mantiene un overhead constante de 33.3% con $m=8$ y con $m=16$ independiente del tamaño del mensaje, dado que agrega r bits de paridad por cada bloque de m bits y su costo escala de forma proporcional. Por el contrario CRC-32 añade siempre 32 bits de modo que su costo se amortiza conforme crece el mensaje, este pasa de un 87.5% en mensajes de 8 bits a apenas un 1.5% en mensajes de 2048 bits.





Discusión:

Cuando la información viaja por un medio físico siempre existe la posibilidad de que algún bit llegue alterado. La pregunta no es cómo evitarlo, sino qué hacer cuando ocurre, para esto hay 2 respuestas: detectar el daño y pedir que el mensaje se reenvía o corregirlo en el destino sin retransmitir. Ambas exigen enviar información adicional, pero la aprovechan de forma distinta. Sirve la analogía de un paquete con una lista de contenido, si la lista solo dice “van diez artículos” quien recibe nota que falta uno pero no puede reponerlo, si describe cada artículo, puede reconstruir el faltante. Esa diferencia separa a CRC-32 de los código de Hamming. Como señalan Tanenbaum y Wetherall (2011), la elección entre ambos enfoques no responde a una superioridad técnica sino a las condiciones del canal y al costo de retransmitir.

En la sección de resultados puse a prueba 5 casos que recorren el rango completo de comportamientos. Sin ruido que corresponde al 5to caso la arquitectura entrega el mensaje sin alterar un bit, lo que confirma que el problema está en el canal y no en la implementación. Con un error, Hamming lo localiza y lo repara como en el 1er caso. Con errores en bloques distintos corrige ambos de forma independiente como sucede en el 2do caso, lo que evidencia que la corrección por bloques escala con la longitud del mensaje. El 3er caso expone la limitación de CRC-32 porque detecta el daño con certeza pero descarta la trama, pues no dispone de información para reconstruirla. El 4to caso marca el límite de la corrección con 2 errores en un mismo bloque, el síndrome (15) excede el tamaño del bloque ($n=12$) y el receptor se declara incapaz.

Cambiando el enfoque dentro del mismo tema vale la pena resaltar que algoritmo tuvo mejor funcionamiento, para esto la respuesta depende del criterio y los datos obligan a matizar. Por cantidad de mensajes entregados correctamente Hamming es superior en canales de buena calidad con una tasa de error de 0.001 entregó el 98.3% frente al 15.0% de CRC-32, diferencia que se explica porque este último descarta toda la trama dañada por leve que sea el daño. Pero si el criterio es la confiabilidad de lo entregado, la conclusión cambia, porque en 10,500 transmisiones de CRC-32 no entregó ni una sola vez datos corruptos como válidos, mientras Hamming lo hizo en el 44.7% de los casos con una tasa de 0.01. Hamming entrega más mensajes pero no todos son confiables. Por el contrario CRC-32 entrega menos pero lo que entrega es confiable. La distinción es central en el diseño de sistemas de comunicación, donde un error no detectado suele resultar más costoso que una retransmisión (Stone et al., 1998).

En detección CRC-32 es incomparablemente más robusto, ya que mantuvo un 0% de errores no detectados en todo el barrido, incluso con tasas de 0.1 donde ninguna trama llegaba intacta. Esto se apoya en una propiedad demostrada por Peterson y Brown (1961), según la cual un código cíclico con polinomio generador de grado r detecta la totalidad de las ráfagas de longitud menor o igual a r . Con $r = 32$, toda ráfaga de hasta 32 bits consecutivos se detecta con certeza, eso lo verificamos de forma intensa en las pruebas unitarias que hicimos en el código. Koopman (2002) estima que la probabilidad de que un patrón arbitrario pase inadvertido es del orden 2^{-32} . Hamming en cambio tiene un umbral que es más rígido, su distancia mínima es 3 lo que según el resultado clásico de Hamming (1950) permite corregir un error o detectar 2, pero no ambas cosas a la vez.

Luego tenemos algo que mostramos en los programas que es el overhead, este se comporta de forma opuesta en ambos esquemas. Hamming mantiene una proporción constante de aproximadamente 33.3% con $m=8$ y 26.1% con $m=16$ porque agrega r bits por cada bloque m , de modo que su costo escala con el mensaje. CRC-32 añade siempre 32 bits, así que su costo relativo disminuye del 87.5% en mensajes de 8 bits al 1.5% en mensajes de 2048, con un punto de equilibrio cercano a los 128 bits. Por eso los protocolos reales, Ethernet entre ellos emplean CRC sobre tramas grandes, la verificación resulta despreciable frente al volumen transportado (IEEE, 2022). En cuanto al cómputo, ambos escalan linealmente.

Por último de los resultados surge un criterio del cual vale la pena hablar, y es la detección con retransmisión conviene cuando existe un canal de retorno y retransmitir es barato, la situación habitual en redes cableadas y locales, CRC-32 ofrece la máxima garantía de integridad con mínimo overhead y las tramas dañadas simplemente se reenvían, que es la estrategia de Ethernet y de los protocolos de transporte de internet (Kurose y Ross, 2021). La corrección conviene cuando retransmitir es imposible, muy costoso o demasiado lento por

ejemplo en comunicaciones satelitales y con sondas espaciales, donde la latencia se mide en minutos, luego medios de almacenamiento donde no hay emisor al cual pedir el dato nuevo y transmisiones en tiempo real donde un paquete retransmitido llegaría tarde (Tanenbaum y Wetherall, 2011). Ambos enfoques no son excluyentes y los sistemas reales los combinan para corrección de errores frecuentes de un bit y por encima de detección que garantice que nada corrupto llegue a la aplicación. Esa combinación resuelve precisamente el problema que evidenció la gráfica 2, pues aporta a la corrección la garantía de integridad de que la que carece por sí sola.

Comentario grupal sobre el tema:

Al iniciar este laboratorio dábamos por echo que la corrección de errores era simplemente que un algoritmo repare el daño, parecía algo evidente que resultaba preferible a uno que se limita a señalarlo. Los resultados nos obligaron a cambiar la perspectiva del tema. Por ejemplo al entender más a fondo a Hamming y que este llegaba a entregar mensajes corruptos en un 44.7% de los casos afirmando haberlos corregido, mientras que CRC-32 no falló ni una sola vez en más de 10,000 transmisiones, nos hizo comprender que la pregunta importante en todo esto es ¿Puede este algoritmo arreglar errores? sino ¿Sabe reconocer cuándo no pudo? Un sistema que falla de forma clara resulta más confiable que uno que falla en silencio y esa idea trasciende el ámbito de las redes.

También nos resultó valioso implementar el emisor y receptor en lenguajes distintos. Al principio lo vimos como una restricción random que alguien se había inventado pero terminó mostrándonos algo valioso y es que un protocolo no es documentación, es un contrato del que dependen 2 programas que no comparten una sola línea de código. Definir el formato de trama antes de escribir nada y verificar ambas implementaciones contra un mismo conjunto de casos de referencia, fue lo que permitió que la integración funcionara sin sorpresas. Comprendimos que los estándares de red son tan claros y detallados porque son la única manera de sistemas contruidos por diferentes personas logren comunicarse entre sí

Conclusiones:

- Ningún algoritmo resultó superior en términos absolutos, la elección depende del criterio. Como se mencionó antes, Hamming entregó 98.3% de los mensajes con una tasa de error del 0.01 frente al 15.0% de CRC-32, pero fue este último el único que nunca entregó información corrupta. Hamming recupera más mensajes, CRC-32 garantiza que lo recuperado sea confiable.
- La detección demostró ser más robusta que la corrección. CRC-32 mantuvo un 0% de errores no detectados en las 10,500 transmisiones evaluadas, incluso con tasas de error de 0.1, mientras que Hamming alcanzó un 44.7%. La capacidad de detección no se degrada con el ruido, la de corrección se agota en cuanto a un bloque acumula más de un error.
- El mayor riesgo no se encuentra en los canales peores, sino en los intermedios. Los errores no detectados de Hamming alcanzan su máximo con una tasa de 0.01 y desaparecen en tasas superiores, donde el algoritmo reconoce su incapacidad en lugar de equivocarse. Un canal moderadamente ruidoso resulta más peligroso que uno claramente deficiente, resultado contrario a la intuición inicial.
- El criterio práctico de selección combina el overhead y la disponibilidad de un canal de retorno. CRC-32 amortiza sus 32 bits constantes hasta un 1.5% en mensajes de 2048 bits, frente al costo fijo de entre 23.8% y 42.9% de Hamming, por lo que conviene siempre que sea posible retransmitir. La corrección se justifica únicamente cuando la retransmisión es inviable, como enlaces satelitales o medios de almacenamiento.

Citas y Referencias:

- Hamming, R. W. (1950). Error detecting and error correcting codes. The Bell System Technical Journal, 29(2), 147–160. <https://doi.org/10.1002/j.1538-7305.1950.tb00463.x>
- Institute of Electrical and Electronics Engineers. (2022). IEEE Standard for Ethernet (IEEE Std 802.3-2022). IEEE. <https://doi.org/10.1109/IEEESTD.2022.9844436>
- Koopman, P. (2002). 32-bit cyclic redundancy codes for Internet applications. En Proceedings International Conference on Dependable Systems and Networks (pp. 459–468). IEEE. <https://doi.org/10.1109/DSN.2002.1028931>
- Kurose, J. F., y Ross, K. W. (2021). Computer networking: A top-down approach (8.^a ed.). Pearson.
- Lin, S., y Costello, D. J. (2004). Error control coding (2.^a ed.). Pearson Prentice Hall.
- Peterson, W. W., y Brown, D. T. (1961). Cyclic codes for error detection. Proceedings of the IRE, 49(1), 228–235. <https://doi.org/10.1109/JRPROC.1961.287814>
- Stone, J., Greenwald, M., Partridge, C., y Hughes, J. (1998). Performance of checksums and CRCs over real data. IEEE/ACM Transactions on Networking, 6(5), 529–543. <https://doi.org/10.1109/90.731187>
- Tanenbaum, A. S., y Wetherall, D. J. (2011). Computer networks (5.^a ed.). Prentice Hall.